

α, β -CROWN을 이용한 MNIST 및 FashionMNIST 모델 강건성 검증

I. 실험 setup

- FashionMNIST

model	Conv(1→16,4×4,s2)-ReLU-Conv(16→32,4×4,s2)-ReLU-FC(1568→100)-ReLU-FC(100→10)
model acc	90.84%
dataset	FashionMNIST
verification property	$\epsilon \in \{1, 2, 4, 6, 8, 12, 16\} / 255$

- MNIST

model	FC(784→32)-ReLU-FC(32→16)-ReLU-FC(16→10)
model acc	95.65 %
dataset	MNIST
verification property	$\epsilon \in \{0.01, 0.03, 0.05, 0.1, 0.2\}$ (과제 #3과 동일)

test sample : test set의 첫 50개 sample

timeout : 인스턴스당 120초

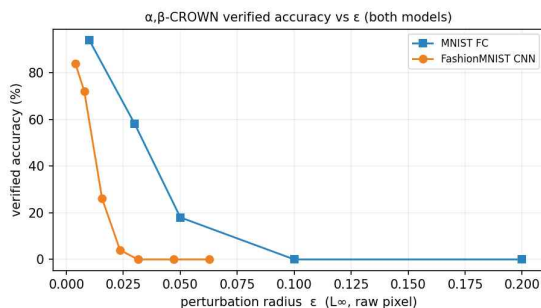
사용한 하드웨어 : RTX 6000 Ada (48 GB) GPU

II. results

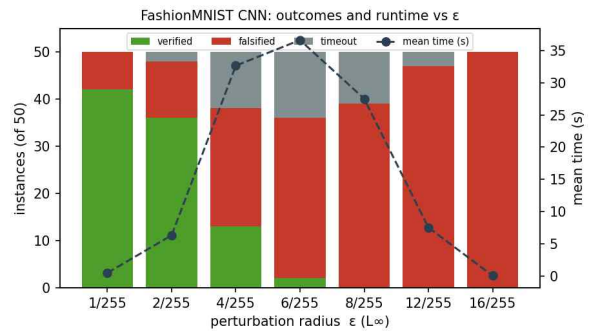
i. FashionMNIST

FashionMNIST CNN에 대해서는 $\epsilon = 1/255$ 부터 $\epsilon = 16/255$ 까지의 perturbation radius를 사용하였다. 각 설정마다 테스트셋의 첫 50개 instance를 검증하였다.

ϵ	Verified	Falsified	Timeout	Mean Time	Max Time
1/255	42	8	0	0.51s	4.54s
2/255	36	12	2	6.32s	125.1s
4/255	13	25	12	32.64s	131.2s
6/255	2	34	14	36.61s	134.1s
8/255	0	39	11	27.51s	132.2s
12/255	0	47	3	7.54s	126.1s
16/255	0	50	0	0.06s	0.64s



ϵ 이 증가할수록 verified instance 수가 감소하였다. (1/255)에서는 50개 중 42개가 verified 되었고, (2/255)에서도 36개가 verified 되었다. 그러나 (4/255)부터 verified 수가 13개로 급격히 감소하고 timeout이 12개 발생하였다. (6/255)에서는 verified가 2개로 줄고 timeout이 14개까지 증가하였다. (8/255) 이후부터는 verified instance가 없었으며, 대부분 falsified 또는 timeout으로 판정되었다.



runtime은 (4/255), (6/255), (8/255) 구간에서 평균 시간이 각각 32.64초, 36.61초, 27.51초로 크게 증가했고, 최대 시간도 약 130초 수준까지 증가하였다. 반면 (16/255)에서는 모든 instance가 falsified 되었고 평균 시간은 0.06초에 불과하였다. 이는 큰 perturbation radius에서는 PGD가 매우 쉽게 counterexample을 찾기 때문에 complete verification까지 갈 필요가 거의 없었기 때문으로 생각된다.

가장 어려운 검증 구간은 가장 작은 ϵ 이나 가장 큰 ϵ 이 아니라, 그 중간 영역이었다.

ii. MNIST

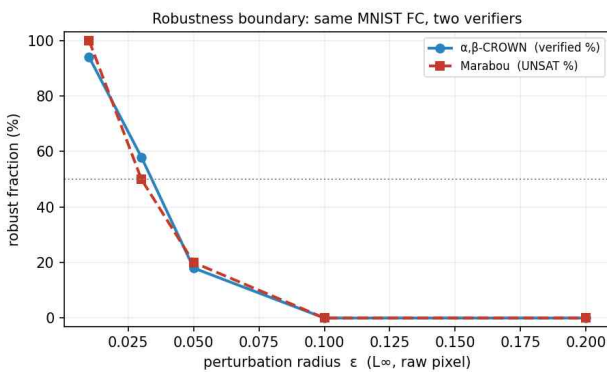
ϵ	Verified	Falsified	Timeout	Mean Time	Max Time
0.01	47	3	0	0.35s	1.08s
0.03	29	21	0	0.38s	1.86s
0.05	9	41	0	0.22s	2.83s
0.1	0	50	0	0.04s	0.59s
0.2	0	50	0	0.03s	0.51s

MNIST FC 모델에서도 ϵ 이 증가할수록 verified instance 수가 급격히 감소함을 볼 수 있었다. $\epsilon=0.01$ 에서는 50개 중 47개가 verified 되었지만, $\epsilon=0.03$ 에서는 29개, $\epsilon=0.05$ 에서는 9개만 verified 되었다. $\epsilon=0.1$ 과 $\epsilon=0.2$ 에서는 모든 instance가 falsified 되었다. 따라서 이 모델의 local robustness boundary는 대략 $\epsilon=0.03$ 과 $\epsilon=0.05$

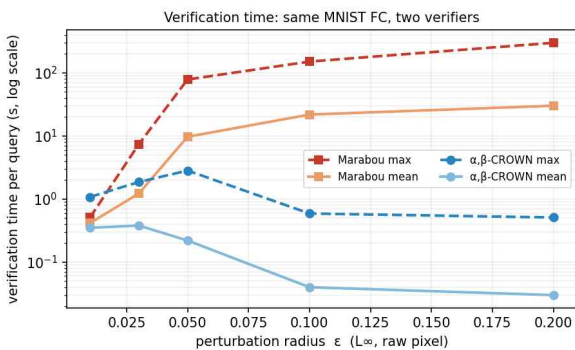
사이에 있다고 해석할 수 있다.

FashionMNIST CNN과 달리 MNIST FC에서는 모든 ϵ 설정에서 timeout이 발생하지 않았다. 또한 평균 시간은 모든 경우에서 0.4초 이하였고, 최대 시간도 2.83초 이하였다. 이는 MNIST FC 모델이 작은 fully-connected ReLU network이기 때문에, α -CROWN의 incomplete verification이나 얇은 β -CROWN branch-and-bound만으로도 대부분의 instance를 빠르게 판정할 수 있었기 때문으로 보인다. 즉, 데이터셋이 아니라 모델의 크기와 안정성이 검증 비용을 좌우함을 확인할 수 있었다.

III. Marabou와의 비교



MNIST FC 모델은 Assignment #3에서 Marabou로 검증했던 모델과 동일한 구조를 사용하였다. Marabou의 robust fraction은 ($\epsilon=0.01, 0.03, 0.05, 0.1, 0.2$)에서 각각 100%, 50%, 20%, 0%, 0%였다. α, β -CROWN은 같은 ϵ 값에서 각각 94%, 58%, 18%, 0%, 0%의 verified accuracy를 보였다. 두 실험의 sample 수가 완전히 같지는 않지만, 같은 architecture와 같은 ϵ 값에서 매우 유사한 robustness boundary를 보였다.



runtime에서 많은 차이를 보였다. Marabou는 $\epsilon=0.05$ 에서 평균 9.70초, $\epsilon=0.1$ 에서 평균 21.85초, $\epsilon=0.2$ 에서 평균 30.16초가 걸렸고, 최대 시간은 300.33초까지 증가하였다. 반면 α, β -CROWN은 MNIST FC 전체 설정에서 평균 0.4초 이하, 최대 2.83초 안에 판정을 완료했다. 두 도구가 비

슷한 robustness conclusion을 낸 것과 다르게, 시간에서 큰 차이를 보였으며, α, β -CROWN이 훨씬 빠르고 안정적으로 동작하였다.

IV. α, β -CROWN의 강점과 한계

i. 강점

α, β -CROWN의 가장 큰 강점은 빠른 검증 속도와 효율적인 pipeline 구조이다. MNIST FC 실험에서 α, β -CROWN은 Marabou와 유사한 robustness boundary를 보였지만 훨씬 빠르게 판정하였다. 예를 들어 $\epsilon=0.05$ 에서 α, β -CROWN은 평균 0.22초, 최대 2.83초 만에 50개 instance를 모두 판정했지만, Marabou는 같은 ϵ 에서 평균 9.70초, 최대 78.56초가 걸렸다. 이는 PGD attack으로 쉬운 반례를 먼저 찾고, α -CROWN으로 쉽게 증명되는 instance를 빠르게 처리한 뒤, 어려운 경우에만 β -CROWN branch-and-bound를 수행하기 때문으로 해석된다.

또한 YAML configuration 기반이라 여러 ϵ 값을 바꿔가며 실험하기 편리했다. 본 실험에서도 MNIST FC와 FashionMNIST CNN에 대해 여러 perturbation radius를 각각 YAML config로 관리해서 실험했다.

ii. 한계

첫째, 어려운 instance에서는 branch-and-bound 과정에서 subdomain 수가 급격히 증가해 메모리 부담이 커진다. 실제로 FashionMNIST CNN의 $\epsilon=8/255$ 실험에서 solver batch size가 클 때 GPU OOM이 발생하여, 이후 batch size를 낮춰 실험을 진행하였다.

둘째, robustness boundary 근처의 ϵ 에서는 timeout이 발생할 수 있다. 본 실험에서도 FashionMNIST CNN의 (4/255), (6/255), (8/255)에서 각각 12개, 14개, 11개의 timeout이 발생하였다.

셋째, 검증 결과는 ϵ , normalization, input range 정의가 정확히 맞을 때만 의미가 있다. 특히 FashionMNIST CNN은 mean/std normalization을 사용했기 때문에, pixel space의 ϵ 이 정규화 공간에서 올바르게 변환되도록 설정해야 했다.